INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

Assignment 03: Project AeroBrain

Grounding Your Assistant in Real Documents

Course: Agentic AI: From Concepts to Practice
Instructor: Prof. Karthik Vaidhyanathan
TAs: Aviral Gupta, Aneetta Sara Shany, Ch Pavan Harshit, Shreyash Chandak

Topic: Retrieval-Augmented Generation (RAG)
Deadline: 9th August 2026, 11:59 PM

Mission Briefing

TO: Lead AI Engineer (formerly AI Systems Engineer)
FROM: Head of Operations, AeroWing Airlines
SUBJECT: The "Hallucination" Incident

AeroAssist has been running in the support centre for two weeks and the team likes it. The interface works, the routing between the cloud and the local model works, and the system prompt keeps the tone professional. However, we have a serious problem.

Yesterday a pilot asked the assistant, *"What is the maximum ramp weight of the A320?"* The assistant confidently replied, *"It is 500,000 kg"* — a figure that is physically impossible for that airframe. In a separate conversation it informed a customer that the A320 cargo hold features a heated swimming pool. Both answers were delivered in the same calm, professional tone as the correct ones.

The Diagnosis: The assistant is *hallucinating*. It is answering from general internet training data instead of from *our* aircraft documentation. In the last assignment you pasted a short quick-reference note into the prompt and the answers improved immediately — but that note was half a page. Our real manuals run to hundreds of pages and cannot be pasted into every request.

Your Mandate: Upgrade AeroAssist into **AeroBrain**. Instead of guessing, the assistant must look up the relevant pages of the official manual *before* it answers, and it must tell us when the manual does not contain the answer. A confident wrong number is worse than an honest "I do not know."

Learning Objectives

By the end of this assignment you should be able to

- Explain why long documents cannot simply be pasted into a prompt, and what chunking solves.

- Convert text into embeddings and store them in a vector database.
- Implement semantic retrieval using cosine similarity and inspect what was retrieved.
- Inject retrieved context into a prompt so that the model answers strictly from the source.
- Make a model refuse gracefully when the answer is genuinely not in the document.
- Extend an existing Flask application into a complete retrieval-augmented chatbot.
- Evaluate a RAG system and measure whether grounding actually reduced hallucination.

Assignment Tasks

Part 1: Building the Knowledge Base

Objective: Turn a large technical PDF into small pieces that a language model can work with.

A language model can only read what fits in its context window. Our first job is to break the manual into pieces small enough to send, but large enough to still make sense on their own.

1. **Download the Manual.** In your Colab notebook, use `wget` to download the official **Airbus A320 Airport Planning Manual**.
   Target URL: `https://www.aircraft.airbus.com/sites/g/files/jlcbta126/files/2025-01/AC_A320_0624.pdf`
2. **Extract the Text.** Use a PDF library such as `pypdf` to extract the text of the document.
   - This document is large. You may restrict your extraction to a portion of it — Chapter 2 (Aircraft Description) is a good choice and contains most of the specifications support executives are asked about.
   - Print the number of characters you extracted, and print the first 1000 characters to confirm the extraction actually worked. PDF text extraction is often messier than people expect.
3. **Chunking.** Write a function `chunk_text(text, size, overlap)` that splits the extracted text into chunks of a fixed number of characters, with a fixed overlap between consecutive chunks. Start with a size of 5000 and an overlap of 500.
   - Report how many chunks you produced.
   - **Why overlap?** A sentence that falls exactly on a chunk boundary would otherwise be split in half, and neither half would answer the question. The overlap ensures that sentences near a boundary appear complete in at least one chunk.
4. **Inspect Your Chunks.** Print any two consecutive chunks and confirm that the overlap is present. Answer in a markdown cell: what would go wrong if you set the chunk size to 50 characters? What would go wrong if you set it to 200,000?

Part 2: Embeddings and the Vector Store

Objective: Convert text into numbers so that meaning can be compared mathematically.

Keyword search fails when the user and the manual use different words for the same thing. A user asks about "legroom"; the manual says "seat pitch". Embeddings solve this by mapping

text to vectors where similar meanings land close together.

1. **Generate Embeddings.** Use the Gemini embedding model through the `google-generativeai` library to generate an embedding for every chunk from Part 1.
   - Print the length of a single embedding vector. How many dimensions does it have?
   - Embed the chunks in batches and be mindful of rate limits.
2. **A Quick Sanity Check.** Embed the three short phrases below and compute the cosine similarity between each pair. Do the numbers match your intuition about which two are most alike?
   - "What is the wingspan of the aircraft?"
   - "How wide is the plane from wingtip to wingtip?"
   - "What is the baggage allowance for economy passengers?"
3. **Build the Vector Store.** Store the chunk embeddings in a vector database. You may use either **FAISS** or **ChromaDB**. Keep the original chunk text alongside each vector — you will need to send the text, not the vector, to the model.

Part 3: The Retrieval Engine

Objective: Find the right page of the manual for a given question.

1. **The Retrieval Function.** Write a function `retrieve_manual(query, k=3)` that
   - converts the user's query into an embedding using the same embedding model,
   - compares it against every stored chunk using **cosine similarity**,
   - returns the **top** k most similar chunks together with their similarity scores.
   - *Hint:* your vector database can do most of this work for you. Understand what it is doing before you let it do it.
2. **Test the Retriever Alone.** Before involving any language model, run these queries through `retrieve_manual` and read the chunks that come back:
   - "What is the wingspan of the A320?"
   - "How much does the aircraft weigh when empty?"
   - "What is the CEO's name?"

   For each, state whether the retrieved chunks actually contain the answer. Pay close attention to the third one: what similarity scores did it return, and what does that tell you about a retriever that always returns something?
3. **Tuning Experiment.** Rebuild your index with a smaller chunk size (for example 1000 characters with 100 overlap) and re-run the same three queries. Record what changed in a markdown cell. Which configuration retrieved more precise passages, and which one gave the model more surrounding context to work with?

Exploration (Not Graded)
Try varying k. With $k = 1$ the model sees one passage and little else; with $k = 10$ it sees a great deal of text, much of it irrelevant. Does a larger k always produce better answers? What does it cost you in tokens and latency?

Critical Incident Report

The retrieval engine is working. On a test question about wingspan, the correct passage was returned as the top result — and the assistant *still* produced a number that does not appear anywhere in that passage.

Retrieval alone is not grounding. The passage has to be placed in front of the model with instructions strict enough that the model prefers the document over its own memory, and honest enough that it says nothing when the document says nothing.

Part 4: Grounded Generation and Prompt Engineering

Objective: Force the model to answer from the retrieved context, and only from it.

1. **Write the System Prompt.** Building on the system prompt you wrote in Assignment 02, define the role, the constraint, and the tone. Some samples are given below — you do not need to follow them exactly.
   - *Role:* "You are an AeroWing Technical Operations Assistant."
   - *Constraint:* "Answer using only the Context Information provided below. If the answer is not present in the context, reply exactly: 'Data not available in manual.' Do not guess and do not use outside knowledge."
   - *Tone:* "Be precise, concise and professional."
2. **Context Injection.** Write a function `build_prompt(query, chunks)` that assembles the final prompt sent to the model. It should follow this structure:

```
Context Information:
{Insert Retrieved Chunks Here}

User Question:
{Insert User Query Here}

Instructions:
Answer the question based strictly on the context above.
If the context does not contain the answer, say so.
```

3. **Ask the Same Question Three Ways.** Take one factual question that the manual can answer, and record the response when it is sent
   - with no context at all,

- with three randomly chosen chunks as context,
- with the three retrieved chunks as context.

The middle case matters. It shows whether your improvement came from retrieval or merely from the presence of some text in the prompt.

4. **The Refusal Test.** Ask a question the manual cannot possibly answer, such as "What is the in-flight Wi-Fi password?" Does your assistant refuse cleanly, or does it invent an answer? If it invents one, revise your system prompt until it refuses reliably. Record both the failing prompt and the working prompt in your notebook.

Part 5: Deploying AeroBrain

Objective: Wire retrieval into the chatbot you already built.

You are not starting from a blank notebook. Take your AeroAssist application from Assignment 02 and upgrade it.

1. **Update the Chat Endpoint.** Modify your Flask `/chat` route so that, for every incoming message, it
   - calls `retrieve_manual(user_message)` to fetch the relevant passages,
   - calls `build_prompt` to construct the grounded prompt,
   - sends that prompt, together with your system prompt, through the `ask` routing function you wrote in Assignment 02, so that RAG works with both the cloud model and the local Ollama model.
2. **Add a RAG Toggle.** Add a configuration variable (and, if you like, a checkbox in the interface) that turns retrieval on and off, so a support executive can compare the grounded and ungrounded answer to the same question. Nothing else in the application should need to change when it is flipped.
3. **Show the Sources.** Update your frontend so that the retrieved context is displayed below each answer — collapsed by default is fine. An operations officer must be able to check which part of the manual an answer came from before acting on it.
4. **Handle the Slow Path.** Retrieval plus generation takes noticeably longer than generation alone. Add a simple "thinking" indicator so the interface does not appear frozen while the request is in flight.

Exploration (Not Graded)
Try running the whole pipeline locally: a local embedding model in Ollama together with your local chat model, with no cloud calls at all. How does the answer quality compare, and what would that change about where AeroWing could deploy this system?

Part 6: Evaluation

Objective: Measure whether grounding actually worked.

An engineer does not claim a system is better; an engineer shows it.

1. **Build a Test Set.** Write **six** questions in your notebook:
   - four that the manual can answer (specifications, dimensions, capacities),
   - two that it cannot (company policy, pricing, crew rosters).
2. **Run Each Question Twice**, once with retrieval disabled and once with it enabled, and fill in a table like the one below.

Question	Answer without RAG	Answer with RAG	Grounded?

3. **Count the Failures.** For how many of the six questions did the ungrounded assistant produce a confident but unverifiable answer? How many did the grounded assistant produce? State the numbers plainly.
4. **Find a Case Where RAG Loses.** Every honest evaluation includes one. Find at least one question your system answers *worse* with retrieval enabled — a retrieved passage that misleads it, or a question so general that the manual only muddies it — and explain what went wrong.

Final Report

Answer the following in your notebook. Each answer should be three to five sentences.

1. In Assignment 02 you improved answers by pasting a quick-reference note directly into the prompt. In this assignment you built a retrieval pipeline instead. Describe one situation where the simpler approach is still the better engineering choice.
2. Your retriever always returns its top k chunks, even for a question the manual never addresses. Explain why this is a risk in a production system, and describe one concrete way you could reduce it.
3. AeroWing wants to add the cabin crew handbook, the maintenance manual and the refund policy to the same assistant. What would you need to change in your design, and what new failure modes would you expect once several documents share one index?
4. Which had the greater effect on answer quality in your experiments: improving the retrieval (chunk size, k) or improving the prompt? Support your answer with specific observations from Part 3 and Part 4.

Submission Guidelines

- Submit a single zip file consisting of
  - a Google Colab notebook (`.ipynb`) containing all code and markdown answers,

  - screenshots of your chatbot taken after Part 5, one showing an answer with its sources displayed and one showing the assistant correctly refusing an unanswerable question. (Titled `partX.jpeg` or `partX.png`)

- Ensure your notebook runs from top to bottom without errors after a kernel restart.
- Store your Gemini API key securely using Colab Secrets. **DO NOT** leave your key hard-coded in the notebook.
- Submit the file using the filename:
  `AeroBrain_YourName.zip`

A language model that has read everything still knows nothing about your airline.
Retrieval gives it a source. Good prompting teaches it to stay there.